NAME: ATCHAYA VHARSNE S

REG NO: 192524185

COURE CODE: CSA1515

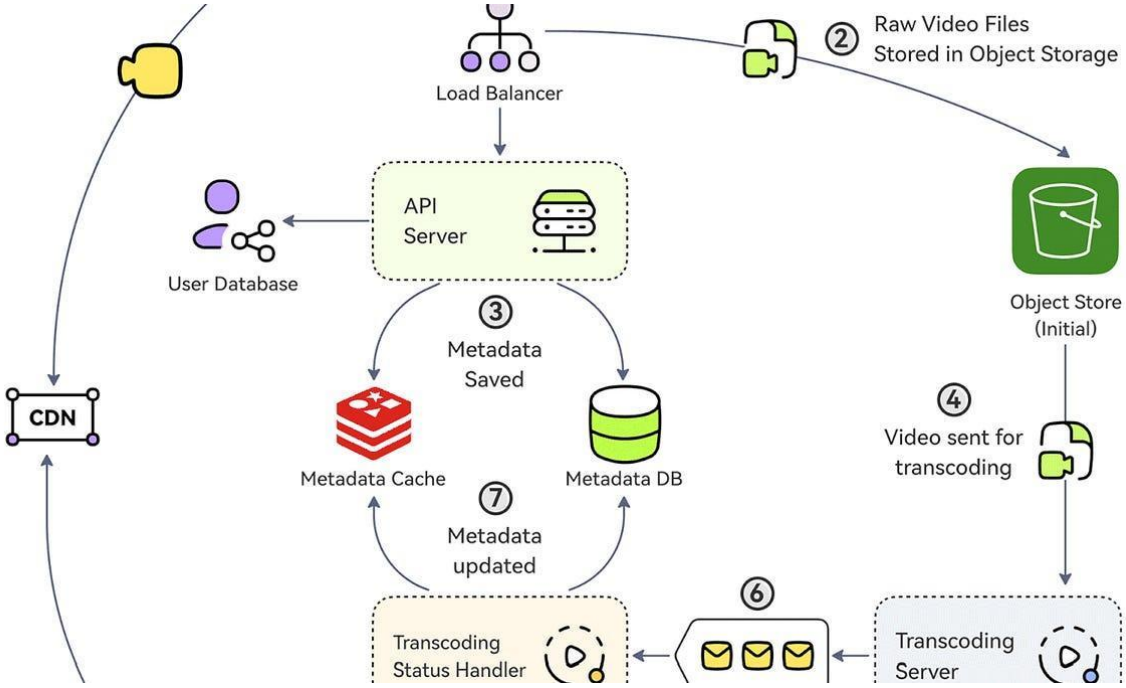
COURSE NAME: CLOUD COMPUTING AND BIG DATA ANALYTICS

CO2 AT2 SCENARIO BASED

Question 1: Cloud Architecture for a Video Streaming Platform

Scenario: A media company wants to migrate its video streaming platform to the cloud. It must handle millions of concurrent users with minimal buffering.

1. Proposed cloud architecture



The proposed architecture consists of:

1. Content Delivery Network (CDN): Stores cached copies of videos at locations closer to users, reducing latency and buffering.
2. Load balancer: Distributes incoming user requests across multiple application servers.
3. Auto-scaling: Automatically increases or decreases the number of servers based on traffic.

4. Object storage: Stores video files, thumbnails and transcoded video versions.
5. Video transcoding: Converts videos into multiple resolutions and bitrates.
6. Database: Stores user accounts, video details and viewing history.
7. Monitoring: Tracks latency, server load, bandwidth and playback errors.

2. Scalability and caching techniques

Scalability allows the system to handle increasing numbers of users.

- Horizontal scaling adds more application server instances.
- Auto-scaling adjusts server capacity according to demand.
- Load balancing distributes traffic across available servers.
- CDN caching reduces repeated requests to the origin server.
- Adaptive bitrate streaming adjusts video quality according to the user's network speed.

3. Load balancing and auto-scaling

The load balancer distributes requests among healthy servers to avoid overloading one server. Auto-scaling adds instances when demand increases and removes unnecessary instances when demand falls.

For example, during a live sports event, the platform can automatically increase capacity to handle a sudden rise in viewers.

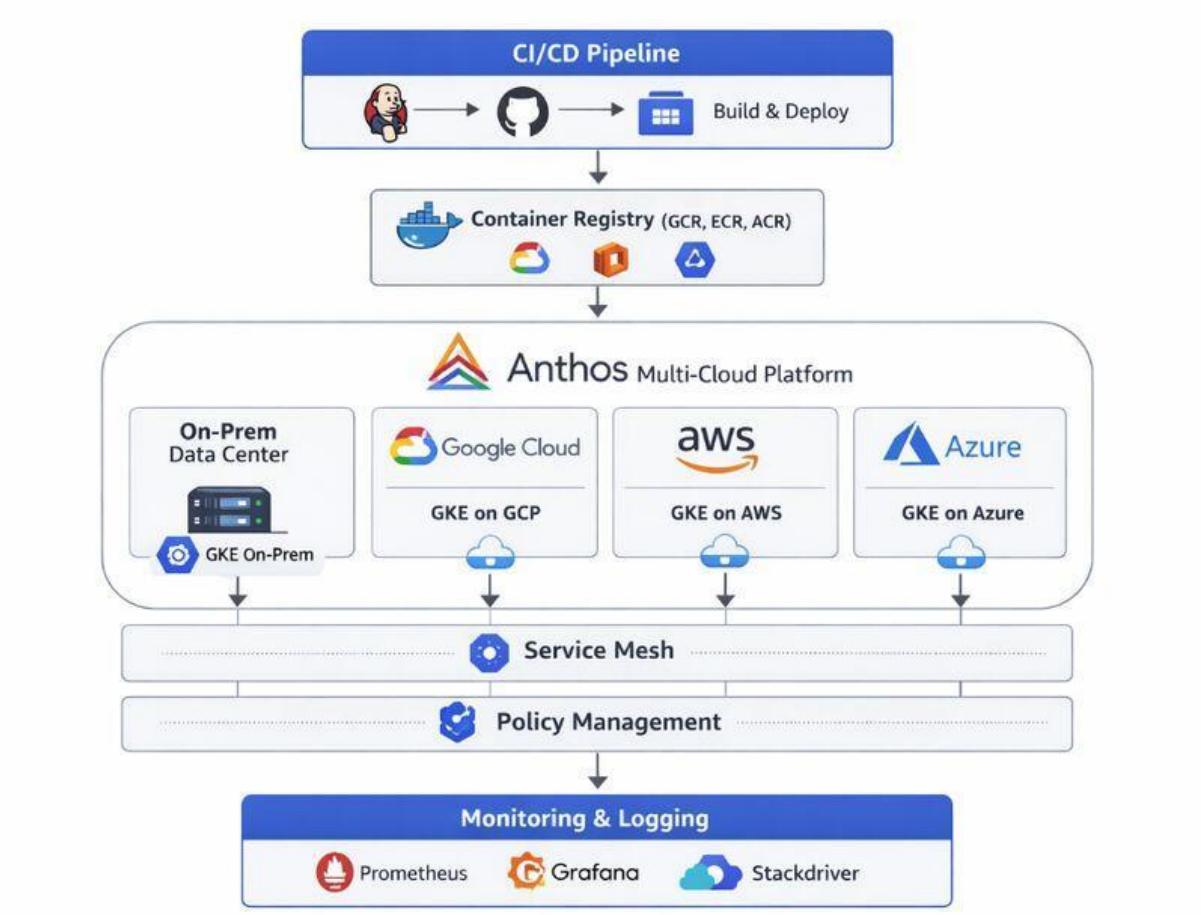
4. Cost optimization

- Use a CDN to reduce origin traffic and repeated data transfers.
- Apply lifecycle policies to move older videos to cheaper storage tiers.
- Use auto-scaling to avoid paying for unnecessary servers.
- Compress and transcode videos into suitable formats.
- Monitor bandwidth, storage and computing costs.

Question 2: Multi-Cloud Container Deployment

Scenario: A fintech startup deploys containerized applications across multiple cloud providers but faces deployment consistency and management challenges.

1. Proposed solution



Use Docker and Kubernetes to package, deploy and manage applications consistently across multiple clouds.

- Docker: Packages an application and its dependencies into a container.
- Kubernetes: Automates container deployment, scaling, networking and recovery.
- Container registry: Stores versioned container images.
- CI/CD pipeline: Automates building, testing and deploying applications.
- Infrastructure as Code (IaC): Tools such as Terraform help provision infrastructure consistently.

2. Managing deployment consistency

1. Package applications using Docker images.
2. Store approved images in a secure container registry.
3. Use Kubernetes manifests or Helm charts to define deployments.
4. Use the same tested image across cloud environments.
5. Automate deployment through a CI/CD pipeline.
6. Monitor application health and deployment status.

This approach reduces configuration differences and manual errors.

3. Service discovery and scaling

Service discovery enables applications to locate other services using service names or registries rather than fixed IP addresses.

Kubernetes Services and DNS support service discovery within a cluster. Cross-cloud communication may require additional networking and service discovery configuration.

Scaling techniques:

- Horizontal Pod Autoscaler increases or decreases application replicas.
- Cluster autoscaling adjusts the number of worker nodes.
- Load balancers distribute requests among application instances.

4. Benefits and risks of multi-cloud deployment

Benefits	**Risks**
Reduces dependence on one provider	More complex management
Improves resilience when designed correctly	Cross-cloud network latency
Provides flexible service choices	Data transfer costs
Supports workload distribution	Different IAM and security systems

Can improve availability	More complex troubleshooting

5. Security measures

- Scan container images for vulnerabilities.
- Store secrets in secure secret-management systems.
- Apply least-privilege access controls.
- Encrypt data in transit and at rest.
- Monitor deployments and maintain audit logs.

6. Conclusion

Docker, Kubernetes, CI/CD and Infrastructure as Code help maintain consistency across multiple cloud providers. Although multi-cloud deployment offers flexibility and resilience, it requires careful management of security, networking, costs and identity policies.

Question 3: Cloud Backup and Disaster Recovery

Scenario: An e-learning platform loses data due to accidental deletion and needs a robust backup and disaster recovery strategy.

1. Proposed solution

The platform should implement a cloud-native backup and disaster recovery system using automated backups, replication, versioning and a separate recovery region.

Main components

1. Automated backups: Create scheduled backups of databases, files and application configurations.
2. Data replication: Maintain copies of important data in another availability zone or region.
3. Versioning: Preserve earlier versions of files so accidentally deleted or overwritten data can be recovered.
4. Immutable backups: Prevent backup data from being modified or deleted during a defined retention period.
5. Disaster recovery environment: Maintain the resources needed to restore services after a major failure.

2. Recovery Point Objective (RPO)

RPO is the maximum acceptable amount of data loss measured in time.

For example, if the RPO is 15 minutes, the organization aims to limit data loss to the most recent 15 minutes of changes.

Frequent backups or appropriate continuous data-protection mechanisms help achieve a low RPO.

3. Recovery Time Objective (RTO)

RTO is the target time within which a system or service must be restored after an interruption.

For example, if the RTO is one hour, the platform should aim to restore its required service within 60 minutes.

A preconfigured recovery environment and automated restoration can help reduce RTO.

Term	Meaning	Example
RPO	Maximum tolerable data loss measured in time	15 minutes
RTO	Target time to restore the service	1 hour

These are illustrative targets; actual values should be based on business requirements.

4. Steps to ensure business continuity

1. Identify critical databases, files and applications.
2. Define RPO and RTO for each critical service.
3. Enable automated backups and file versioning.
4. Replicate important data to a separate failure domain.
5. Protect backups using encryption and restricted access.
6. Create a documented disaster recovery plan.
7. Test restoration and failover regularly.
8. Monitor backup failures and verify backup integrity.

Important: Replication alone is not enough. Accidental deletion or corrupted data may also be replicated, so independent versioned or immutable backups are necessary.

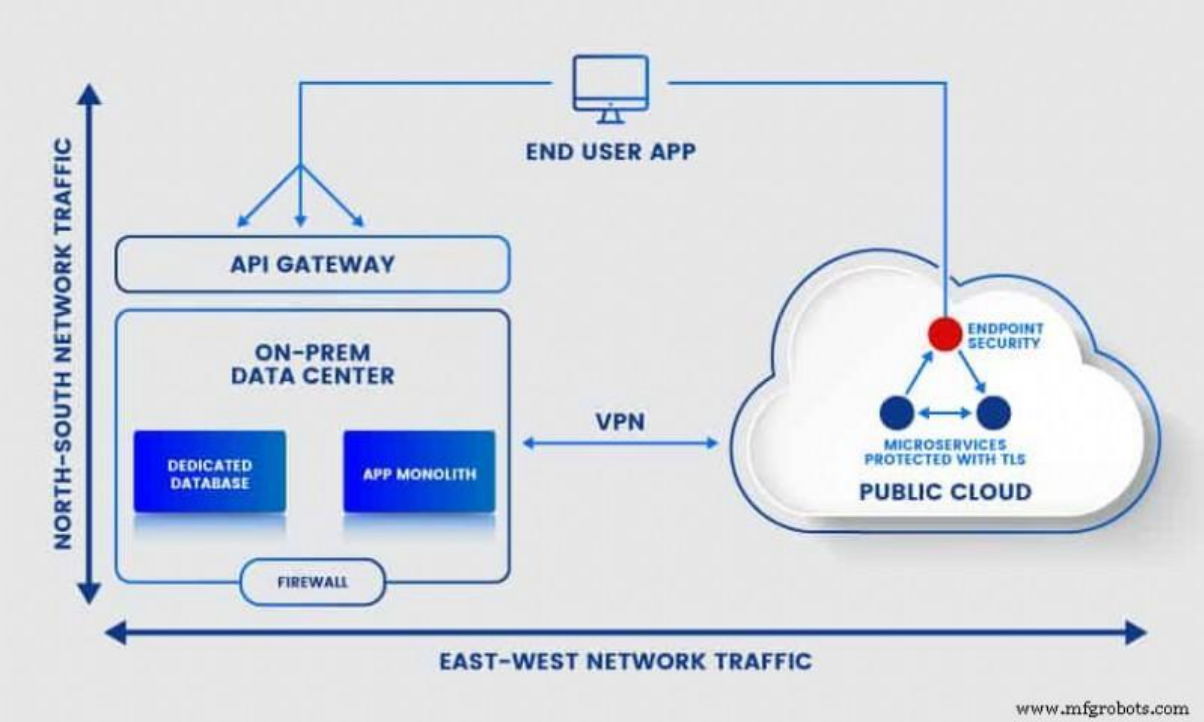
5. Conclusion

A combination of automated backups, replication, versioning and immutable backup storage protects the e-learning platform against accidental deletion and major failures. Clearly defined RPO and RTO targets, along with regular recovery testing, help maintain business continuity and minimize downtime.

Question 4: Hybrid Cloud for a Government Agency

Scenario: A government agency wants to adopt a hybrid cloud for sensitive and non-sensitive data. Some applications must remain on-premises for compliance requirements.

1. Proposed hybrid cloud architecture



A hybrid cloud combines on-premises infrastructure or a private cloud with public cloud services, allowing workloads and data to be placed according to security, performance and compliance requirements.

Architecture components

1. On-premises infrastructure: Hosts highly sensitive applications and data that must remain within agency-controlled facilities.
2. Private cloud: Provides virtualized resources with greater control over access and configuration.
3. Public cloud: Hosts suitable non-sensitive applications, public information portals and scalable services.
4. Secure connectivity: VPN or dedicated private connectivity links the environments.
5. Central identity management: Controls access across on-premises and cloud systems.
6. Monitoring and auditing: Tracks activity and helps detect security violations.

2. Data placement

Data or application	Recommended location	Reason
Highly sensitive government records	On-premises or approved private environment	Greater control and compliance
Internal applications	Private or approved cloud environment	Controlled access
Public information portal	Public cloud	Scalability and public availability
Public website content	Public cloud or CDN	Efficient delivery

The final placement must follow applicable government regulations and data-residency requirements.

3. Identity management

- Use centralized identity management where practical.
- Implement role-based access control (RBAC).
- Enable multi-factor authentication for sensitive systems.
- Apply least-privilege permissions.
- Use federation and single sign-on where supported.
- Review user accounts and permissions regularly.

4. Encryption

Encryption at rest: Protects stored data in databases, disks and backups.

Encryption in transit: Protects information moving between on-premises and cloud systems, for example using TLS and secure VPN connections.

Encryption keys should be securely managed, access-restricted and rotated according to policy.

5. Challenges in hybrid cloud

- Integrating different infrastructure and management tools.

- Maintaining consistent security policies.
- Managing data residency and compliance.
- Controlling connectivity and data transfer costs.
- Monitoring multiple environments.
- Preventing unauthorized access across connected systems.

6. Conclusion

A hybrid cloud allows the government agency to retain sensitive workloads in approved on-premises or private environments while using public cloud services for suitable applications. Centralized identity management, encryption, network segmentation, continuous monitoring and compliance checks help maintain security and privacy.

Question 5: High Availability and Fault Tolerance in a SaaS Platform

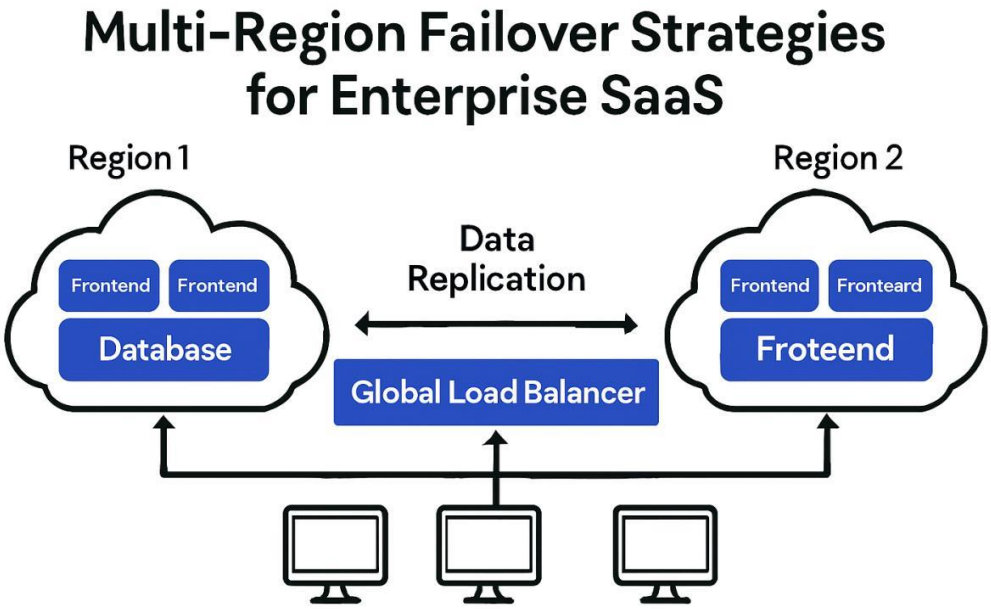
Scenario: A SaaS provider needs high availability and fault tolerance. Its system must continue operating even if a cloud region goes down.

1. Introduction

High availability (HA) aims to minimize service downtime, while fault tolerance aims to keep a system operating despite specified component failures.

A SaaS provider can improve resilience by deploying applications across multiple availability zones and preparing a recovery environment in another region.

2. Proposed architecture



Main components

1. Global load balancer: Routes users to a healthy regional endpoint.
2. Multiple availability zones: Distribute application instances across separate failure domains within a region.
3. Application servers: Run multiple instances so a single instance failure does not stop the service.
4. Database replication: Maintains copies of database data in another zone or region.

5. Health monitoring: Detects failed instances, unhealthy endpoints and regional outages.
6. Disaster recovery region: Provides an alternative environment when the primary region becomes unavailable.

3. Data replication

Data replication copies data from one database or storage system to another.

Two common approaches are:

- Synchronous replication: Confirms changes on the required replicas before completing a write. It can reduce data loss but may increase latency and depends on the database architecture.
- Asynchronous replication: Copies changes after the primary write completes. It generally reduces write latency but may lose recent changes if the primary region fails before replication catches up.

The provider should select the replication strategy according to its RPO, latency and consistency requirements.

4. Failover mechanisms

Failover switches operations from a failed component or region to a healthy alternative.

Steps include:

1. Monitor application and regional health.
2. Detect the failure using health checks and monitoring alerts.
3. Remove unhealthy endpoints from traffic routing.
4. Promote an appropriate database replica if required.
5. Route users to the recovery region.
6. Verify application functionality and data consistency.
7. Restore the original region safely after recovery.

Automated failover must include safeguards against split-brain conditions, where multiple database nodes incorrectly act as the primary.

5. Monitoring and alerting

The SaaS provider should monitor:

- Application response time and error rates.
- CPU, memory and storage utilization.
- Database replication lag.
- Availability-zone and regional health.
- Backup status and recovery readiness.

Alerts should notify administrators when thresholds are exceeded or failover occurs.

Regular disaster recovery drills should verify that the application meets its availability and recovery targets.

6. Conclusion

A multi-region deployment with multiple availability zones, replicated data, health monitoring and automated failover improves SaaS availability and fault tolerance. Synchronous or asynchronous replication should be selected according to the application's consistency and recovery requirements. Regular testing and monitoring help reduce downtime and protect data during regional failures.